

AUC 0.9+ 달성을 위한 모델 개선 방안

근감소증 예측 모델 (Sarcopenia Detection) 연구

Model1 AUC 0.803 → Model2/3 AUC 0.9+ 목표

현황 분석

현재 5개 모델의 성능은 아래와 같으며, AEC 추가 모델이 베이스라인(M1)을 유의미하게 증가하지 못하고 있습니다.

모델	입력 피쳐	아키텍처	Test AUC	AUPRC	문제점
M1	Age / Sex / BMI	Logistic Regression	0.803	0.312	피쳐 3개뿐
M2	Clinic + AEC (matched)	Cross-Attention	0.802	0.346	M1보다 낮음
M2_2	Clinic + AEC (unmatched)	Cross-Attention	0.841	0.354	Negative control 역전
M3	Clinic + Scanner + AEC	3-way CrossAttn	0.830	0.337	유의 향상 없음
M4	AEC Only	ResNet1D	0.604	0.208	AEC 단독 신호 부족

핵심 문제: AEC 시퀀스 raw 사용 시 정보량 부족(M4 AUC=0.60), Cross-Attention 융합이 임상 피쳐 3개를 보완하지 못함. M2_2(unmatched)가 M2(matched)를 능가하는 이상 현상 존재.

개선 방안

1순위 — 피쳐 확장 (가장 빠른 AUC 향상 기대)

[A] AEC Hand-Crafted 피쳐 추출 후 M1에 Concat

M4(AEC Only)의 AUC가 0.60에 불과한 것은 128점 raw 시퀀스를 그대로 신경망에 입력하면 유용한 패턴이 소실됨을 의미합니다. 통계적 요약 피쳐를 추출해 임상 피쳐와 concat하면 deep model 없이도 AUC 향상이 가능합니다.

```
import numpy as np, scipy.stats

def extract_aec_features(aec_seq): # shape: (128,)
    return {
        'aec_mean'      : aec_seq.mean(),
        'aec_std'       : aec_seq.std(),
        'aec_max'       : aec_seq.max(),
        'aec_min'       : aec_seq.min(),
        'aec_peak_pos'  : aec_seq.argmax() / 128, # 정규화 peak 위치
        'aec_auc'       : np.trapz(aec_seq),      # 곡선 아래 면적
        'aec_skew'      : scipy.stats.skew(aec_seq),
        'aec_kurt'      : scipy.stats.kurtosis(aec_seq),
        # 구간별 통계 (early / mid / late attenuation)
        'aec_early_mean': aec_seq[:43].mean(),
        'aec_mid_mean'  : aec_seq[43:86].mean(),
        'aec_late_mean' : aec_seq[86:].mean(),
    }

# 임상 피쳐와 concat → Logistic Regression / XGBoost에 입력
aec_feats = extract_aec_features(aec_seq) # 11개 피쳐
X_combined = np.concatenate([clin_feats, list(aec_feats.values())])
```

추출되는 11개 피쳐의 의미:

- aec_mean / std : 전체 감쇠 수준 및 변동성
- aec_max / min : 피크 감쇠값 및 기저 감쇠값
- aec_peak_pos : 최대 감쇠가 발생하는 시점 (조영제 도달 속도)
- aec_auc : 총 조영제 통과량 (심박출량 대리 지표)
- aec_skew / kurt: 감쇠 곡선의 비대칭도 및 첨도
- early/mid/late : 조영 전기·중기·후기 평균 (동맥상·문맥상·평형상)

기대 효과: 임상 피쳐(3개) + AEC 통계 피쳐(11개) = 총 14개 피쳐로 LR/XGBoost 학습 시 AUC 0.84~0.88 수준 달성 가능 (문헌 유사 연구 기준). 구현이 단순하고 해석 가능성이 높아 논문 기여도도 큼.

[B] Cross-Attention 대신 1D CNN + Late Fusion

현재 Cross-Attention의 문제: AEC 시퀀스(128점) vs 임상 피쳐(3개)의 극단적 차원 불균형으로 attention이 수렴하지 못합니다. 1D CNN으로 지역 패턴을 먼저 압축한 후 임상 피쳐와 late fusion하는 구조가 더 안정적입니다.

```
class ImprovedFusionModel(nn.Module):
    def __init__(self, n_clin=3):
        super().__init__()
        # AEC 인코더: 128 → 지역 패턴 압축
        self.aec_encoder = nn.Sequential(
            nn.Conv1d(1, 32, kernel_size=7, padding=3),
            nn.BatchNorm1d(32), nn.ReLU(),
            nn.MaxPool1d(2),          # 128 → 64
            nn.Conv1d(32, 64, kernel_size=5, padding=2),
            nn.BatchNorm1d(64), nn.ReLU(),
            nn.AdaptiveAvgPool1d(8),  # → 64 x 8 = 512-dim
            nn.Flatten()
        )
        # 임상 피쳐 MLP
        self.clin_encoder = nn.Sequential(
            nn.Linear(n_clin, 32), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(32, 64),    nn.ReLU()
        )
        # Late Fusion 분류기
        self.classifier = nn.Sequential(
            nn.Linear(512 + 64, 128), nn.ReLU(), nn.Dropout(0.4),
            nn.Linear(128, 1)
        )

    def forward(self, aec, clin):
        aec_feat = self.aec_encoder(aec.unsqueeze(1)) # (B, 512)
        clin_feat = self.clin_encoder(clin)           # (B, 64)
        return self.classifier(torch.cat([aec_feat, clin_feat], dim=1))
```

[C] Contrastive Pre-training (데이터가 충분할 경우)

동일 환자의 다회 촬영 AEC를 positive pair로, 다른 환자를 negative pair로 설정하여 SimCLR / SupCon 방식으로 AEC 인코더를 사전학습합니다. 이후 frozen encoder + classifier fine-tuning으로 소규모 레이블 데이터에서도 강건한 표현 학습이 가능합니다.

- 적용 조건: 동일 환자 2회 이상 촬영 데이터 존재 여부 확인 필요
- 대안: AEC augmentation(noise/mask)을 positive pair로 사용하는 self-supervised 방식

구조 개선 기대 효과: 1D CNN Late Fusion → AUC 0.85~0.88 (Cross-Attention 대비 +0.03~0.05).
Contrastive Pre-training → 데이터 크기에 따라 추가 +0.02~0.04.

[D] 하이퍼파라미터 조정

현재 config.py의 설정과 권장 변경값:

파라미터	현재값	권장값	이유
LR_RATE	1e-5	3e-4	너무 낮아 수렴 속도 저하
WEIGHT_DECAY	기본값	1e-3	AdamW 정규화 강화
BATCH_SIZE	32	16	소규모 데이터 시 더 많은 업데이트
EPOCHS	500	300 + ES	Early Stopping으로 과적합 방지
N_HEADS	1	2 또는 4	Multi-head attention 활성화

```
# config.py 권장 수정
LR_RATE = 3e-4 # 1e-5 → 3e-4
WEIGHT_DECAY = 1e-3 # AdamW L2 정규화
BATCH_SIZE = 16 # 소규모 데이터 최적화
N_HEADS = 4 # Multi-head attention

# train_eval.py — Early Stopping 추가
patience, best_val_auc, wait = 30, 0, 0
for epoch in range(EPOCHS):
    ... # train
    if val_auc > best_val_auc:
        best_val_auc, wait = val_auc, 0
        torch.save(model.state_dict(), ckpt_path)
    else:
        wait += 1
        if wait >= patience: break
```

[E] 클래스 불균형 처리 강화

현재 AUPRC=0.31은 심각한 클래스 불균형을 의미합니다. Focal Loss는 이미 사용 중이나 pos_weight가 명시적으로 설정되지 않았을 수 있습니다.

```
# train_eval.py — pos_weight 명시 계산
n_neg = (y_train == 0).sum()
n_pos = (y_train == 1).sum()
pos_weight = torch.tensor(n_neg / n_pos, dtype=torch.float32).to(device)

# FocalLossWithLogits에 전달
criterion = FocalLossWithLogits(gamma=2.0, pos_weight=pos_weight)

# 추가: SMOTE 적용 (선택적)
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE(random_state=42).fit_resample(X_train, y_train)
```

[F] 모델 앙상블

각 모델의 예측 확률을 가중 평균하면 단일 모델 대비 AUC를 0.01~0.03 추가로 향상시킬 수 있습니다. 특히 M1(LR)과 M3(Deep)처럼 서로 다른 귀납 편향을 가진 모델의 앙상블이 효과적입니다.

```
# 테스트 시 예측 확률 앙상블
prob_m1 = model1.predict_proba(X_test)[:, 1] # Logistic Regression
prob_m3 = torch.sigmoid(model3(aec, clin)).cpu().numpy().flatten()

# 가중 평균 (AUC 기반 가중치)
w1, w3 = 0.35, 0.65
prob_ensemble = w1 * prob_m1 + w3 * prob_m3

# 또는 Stacking (Meta-learner)
from sklearn.linear_model import LogisticRegression
meta_X = np.column_stack([prob_m1, prob_m3])
meta_clf = LogisticRegression().fit(meta_X_val, y_val)
```

권장 실행 순서 (로드맵)

Step 1

Excel 원본 컬럼 확인

강남_liver_merged_features_ok.xlsx의 전체 컬럼 목록 확인.
SMI 외 추가 임상/lab 수치 존재 시 CLIN_COLS에 추가.

Step 2

AEC Hand-Crafted 피쳐 추출

extract_aec_features() 구현 → M1(LR)에 concat하여 AUC 확인.
목표: 0.84+ 달성 여부 판단.

Step 3

학습 전략 수정

LR=3e-4, pos_weight 명시, Early Stopping 추가.
기존 M2/M3 재학습 후 성능 비교.

Step 4

1D CNN Late Fusion 모델 구현

ImprovedFusionModel 코드 작성 → 5-Fold CV 비교.
목표: AUC 0.87+ 달성.

Step 5

앙상블로 마무리

Step 2 모델 + Step 4 모델 앙상블.
목표: AUC 0.90+ 달성.

최종 목표 시나리오

Step 1~2 완료 후 AUC 0.85+ 달성 → Step 3~4로 0.87~0.88 → Step 5 앙상블로 0.90+ 목표 달성.
각 Step별 5-Fold CV + DeLong Test로 통계적 유의성 검증 필수.